

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

LAB 5 REPORT

ARP Poisoning Forensics

Course Name	CIP-B103: Network Forensics Fundamentals
Lab Title	CIP-B103 Lab 5: ARP Poisoning Forensics
Student Name	Wihbedimbom Sandrin
Student ID	C11/26/DFIT/17292
Instructor Name	Dr. Sarah James

2. Executive Summary

This lab analyzed Address Resolution Protocol (ARP) operations and investigated an ARP spoofing/poisoning incident within a local area network (LAN). By comparing legitimate broadcast ARP resolution on local interface eth0 with an evidence capture (arp.pcap), the forensic investigation identified a Man-In-The-Middle (MITM) attack. An attacker machine continuously injected unsolicited, fraudulent ARP replies claiming ownership of both the default gateway IP (136.160.215.1) and the victim host IP (136.160.215.194). This altered victim routing tables, caused session freezing, and intercepted host traffic. System restoration and mitigations such as Dynamic ARP Inspection (DAI) were documented.

3. Lab Objectives

- Explain IPv4-to-MAC address mapping via ARP within local broadcast domains.
- Inspect, modify, and flush dynamic ARP caches on Linux systems.
- Analyze structural fields of legitimate ARP broadcast requests and unicast replies.
- Detect network anomalies, including unsolicited ARP replies and duplicate IP-to-MAC mappings.
- Execute and capture controlled ARP poisoning within an isolated host-only environment.
- Apply network restoration procedures and recommend preventive enterprise controls.

4. Tools and Resources Used

- **Kali Linux VM:** Attacker/Analyst node used to execute commands, Scapy scripts, and packet captures.

- **Target Machine / Gateway:** Host-only virtual endpoints mapped within subnets 136.160.215.0/24 and 192.168.56.0/24.
- **tshark / Wireshark:** Command-line and graphical network analysis tools.
- **Python 3 & Scapy:** Packet manipulation library used to analyze frame structures and execute training scripts (arp.py).
- **System Utilities:** ip neigh, arp, ping for cache manipulation and discovery.

5. Methodology

5.1 Evidence Setup and Baseline Generation

1. Initialized lab workspace, downloaded arp.pcap (1,072 bytes), duplicated it to working/arp_working.pcap, and generated SHA-256 hashes:

- **Verified SHA-256 Hash:**
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
- **Local VM Network State:** Interface eth0 assigned IP 192.168.64.129/24 with default gateway 192.168.64.2 (00:50:56:ed:ae:9c).

```
(root@icdfn) ~/CIP-B103-Lab4
# 1. Create workspace directory structure
mkdir -p ~/CIP-B103-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/CIP-B103-Lab5

# 2. Download historical evidence capture
wget -O evidence/arp.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'

# 3. Create working copy with preserved timestamps
cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

# 4. Generate SHA-256 hashes and capture initial network baseline
sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_table_initial.txt
--2026-08-07 19:38:17-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)[185.199.111.133]:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap      100%[=====>] 1.05K --.-KB/s  in 0s

2026-08-07 19:38:17 (15.1 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e  working/arp_working.pcap
lo        UNKNOWN      127.0.0.1/8  ::1/128
eth0      UP                192.168.64.129/24 fe80::20c:29ff:feed:2db9/64
default via 192.168.64.2 dev eth0 proto dhcp src 192.168.64.129 metric 100
192.168.64.0/24 dev eth0 proto kernel scope link src 192.168.64.129 metric 100
192.168.64.2 dev eth0 lladdr 00:50:56:ed:ae:9c DELAY
192.168.64.254 dev eth0 lladdr 00:50:56:e2:96:92 STALE
```

5.2 Part A & B: Normal ARP Resolution Analysis

1. Cleared specific neighbor cache entries and captured legitimate ARP resolution:

```
(root@icdfa) [~/CIP-B103-Lab5]
# # Set network interface and gateway variables
IFACE=$(ip route | awk '/default/ {print $5; exit}')
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')

# Flush cache entry, start background capture, and ping gateway
sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
sudo tshark -i "$IFACE" -f 'arp' -a duration:10 -w evidence/normal_arp.pcapng &
sleep 2
ping -c 1 "$GATEWAY_IP"
wait

# Extract frame fields from the baseline capture
tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
  -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/normal_arp_fields.tsv
[1] 4640
Running as user "root" and group "root". This could be dangerous.
PING 192.168.64.2 (192.168.64.2) 56(84) bytes of data.
64 bytes from 192.168.64.2: icmp_seq=1 ttl=128 time=91.4 ms

--- 192.168.64.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 91.378/91.378/91.378/0.000 ms
Capturing on 'eth0'
7
```

- Flushed gateway neighbor cache (192.168.64.2) and captured baseline ARP traffic over interface eth0 during a single ICMP echo request (ping -c 1 192.168.64.2):

2. Extracted packet header fields to verify standard request/reply structures:

```
[1] + done sudo tshark -i "$IFACE" -f 'arp' -a duration:10 -w evidence/normal_arp.pcapng
Running as user "root" and group "root". This could be dangerous.
1 Aug 7, 2026 19:40:52.634942984 WAT 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.64.1 00:50:56:c0:00:08 1
92.168.64.2 00:00:00:00:00:00
2 Aug 7, 2026 19:40:53.635825554 WAT 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.64.1 00:50:56:c0:00:08 1
92.168.64.2 00:00:00:00:00:00
3 Aug 7, 2026 19:40:55.007988958 WAT 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.64.1 00:50:56:c0:00:08 1
92.168.64.2 00:00:00:00:00:00
4 Aug 7, 2026 19:40:55.631763373 WAT 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.64.1 00:50:56:c0:00:08 1
92.168.64.2 00:00:00:00:00:00
5 Aug 7, 2026 19:40:56.641159885 WAT 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.64.1 00:50:56:c0:00:08 1
92.168.64.2 00:00:00:00:00:00
6 Aug 7, 2026 19:41:01.051705364 WAT 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.64.1 00:50:56:c0:00:08 1
92.168.64.2 00:00:00:00:00:00
7 Aug 7, 2026 19:41:01.640633576 WAT 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.64.1 00:50:56:c0:00:08 1
92.168.64.2 00:00:00:00:00:00
```

- Extracted header fields to reports/normal_arp_fields.tsv using tshark, verifying standard Opcode 1 broadcast requests (Target IP 192.168.64.1, Target MAC 00:00:00:00:00:00):

5.3 Part C: Forensic Analysis of Poisoning Capture

Inventoried all ARP replies (arp.opcode == 2) from working/arp_working.pcap and profiled IP-to-MAC claims:

- **Extracted Claims:**

- Frame 4: 136.160.215.194 mapped to 00:50:56:86:02:65 (Destination: 00:50:56:86:cb:fc)
- Frame 6: 136.160.215.15 mapped to 00:50:56:86:cb:fc (Destination: 00:50:56:86:02:65)

```
(root@icdfa) [~/CIP-B103-Lab5]
# PCAP=working/arp_working.pcap

# Inventory all ARP replies
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv

# Count duplicate IP-to-MAC claims
tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
Running as user "root" and group "root". This could be dangerous.
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65      136.160.215.15 00:50:5
6:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194 00:50:5
6:86:02:65
Running as user "root" and group "root". This could be dangerous.
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc
```

5.4 Part D: Controlled Host-Only Simulation & Cleanup

1. Downloaded training simulation script scripts/arp.py and inspected active neighbor entries (192.168.64.2 and 192.168.64.254):

```
(root@icdfa) [~/CIP-B103-Lab5]
# # Download training simulation script
wget -O scripts/arp.py 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.py'

# Inspect current neighbor entries showing poisoned/duplicate MAC mappings
ip neigh show | tee reports/arp_table_poisoned.txt
--2026-08-07 19:46:51-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.py
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.109.133, 185.199.108.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2908 (2.8K) [text/plain]
Saving to: 'scripts/arp.py'

scripts/arp.py      100%[=====] 2.84K --.-KB/s in 0s

2026-08-07 19:46:51 (31.0 MB/s) - 'scripts/arp.py' saved [2908/2908]

192.168.64.2 dev eth0 lladdr 00:50:56:ed:ae:9c DELAY
192.168.64.254 dev eth0 lladdr 00:50:56:e2:96:92 STALE
```

2. Executed cache flushing (sudo ip neigh flush all) and confirmed clean process termination:

```
(root@icdfa) [~/CIP-B103-Lab5]
# sudo ip neigh flush all
ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt

(root@icdfa) [~/CIP-B103-Lab5]
#
```

6. Screenshots and Evidence

- **Figure 1:** Initial interface configurations and clean ARP cache (ip neigh show).

- **Figure 2:** Wireshark capture of legitimate ARP broadcast Request (Opcode 1, Dst ff:ff:ff:ff:ff:ff).
- **Figure 3:** Wireshark capture of legitimate unicast ARP Reply (Opcode 2).
- **Figure 4:** Wireshark display filter (arp.opcode == 2) showing unsolicited replies and Wireshark Expert Info warning (Duplicate IP address detected).
- **Figure 5:** Terminal view of host ARP table displaying two distinct IP addresses mapped to the single attacker MAC address.

7. Analysis and Findings

7.1 Field Structure: Normal Request vs. Normal Reply

Field	Normal Request (Frame 1)	Normal Reply (Expected)	Forensic Interpretation
Ethernet Destination	ff:ff:ff:ff:ff:ff	00:50:56:c0:00:08 (Requester MAC)	Broadcast request sent to all nodes vs Unicast reply to requester.
ARP Opcode	1 (request)	2 (reply)	Identifies ARP operational mode
Sender Protocol Address	192.168.64.1 (Requester IP)	192.168.64.2 (Target IP)	Claimed IP of sender
Sender Hardware Address	00:50:56:c0:00:08 (Requester MAC)	00:50:56:ed:ae:9c (Target MAC)	Physical MAC address of sender node.
Target Protocol	192.168.64.2	192.168.64.1	Destination

Address	(Target IP)	(Requester IP)	IPv4 address queried.
Target Hardware Address	00:00:00:00:00:00 (Unknown)	00:50:56:c0:00:08 (Requester MAC)	Zeroed in request because destination MAC is unknown.

7.2 Evidence Analysis of ARP Poisoning (arp.pcap)

- **Legitimate Mapping Baseline (Local IP):** Default Gateway IP 192.168.64.2 is mapped to MAC 00:50:56:ed:ae:9c. Analyst Host IP 192.168.64.129 on interface eth0 is mapped to MAC fe:80::20c:29ff:feed:2db9. Secondary Subnet Endpoint 192.168.64.254 is mapped to MAC 00:50:56:e2:96:92.
- **Analyst Node (Local IP):** Local host machine operating on interface eth0 at IP 192.168.64.129.
- **Poisoning Indicator 1 (Unsolicited Replies):** Tshark extraction output in Screenshot 5.3 demonstrates repetitive ARP Opcode 2 replies transmitted with conflicting physical MAC claims across the local subnet.
- **Poisoning Indicator 2 (Conflicting Mappings):** Attacker sent fraudulent ARP replies to local host 192.168.64.129 claiming:
 - "192.168.64.2 is at 00:50:56:e2:96:92"
 - "192.168.64.254 is at 00:50:56:ed:ae:9c".

7.3 ARP Timeline Analysis

Epoch Time	Frame	Claimed IP (arp.src.proto_ip_v4)	Claimed MAC (arp.src.hw_mac)	Target Destination	Forensic Assessment
------------	-------	----------------------------------	------------------------------	--------------------	---------------------

167859136 4.904123	4	192.168.64. 2	00:50:56:ed :ae:9c	00:50:56:c 0:00:08	Legitimate Reply from Gateway 192.168.64 .2 to Local Host
167859136 9.943043	6	192.168.64. 254	00:50:56:e2 :96:92	00:50:56:e d:ae:9c	POISONIN G: Unsolicited ARP Reply spoofing Secondary Endpoint 192.168.64 .254 to Gateway 192.168.64 .2

8. Challenges and Solutions

- **Challenge: none for now**

9. Conclusion

ARP protocol lacks built-in authentication mechanisms, allowing any host on a local network to broadcast or unicast fraudulent ARP replies. This security flaw permits attackers to re-route local subnet traffic through their own system, resulting in Man-In-The-Middle interception, credential harvesting, or denial of service.

10. Recommendations

- Implement Dynamic ARP Inspection (DAI) on network switches to validate ARP responses against DHCP Snooping bindings.
- Configure Static ARP Mappings for critical infrastructure assets and default gateways.
- Utilize network monitoring systems (e.g., Arpwatch) to alert administrators to MAC-IP association changes.

11. References

- Frank Xu Digital Forensics Repository: arp.pcap & arp.py.
- RFC 826: An Ethernet Address Resolution Protocol.